Assignment 3 Design Rationale

REQ 1

Design rationale

Feature
A new area called Limveld got added to the game. It connects to the Valley of the Inheritree. Players can travel between these two maps using TeleportationCircle(A), Stepping onto a ‘A’ makes a TeleportAction available to the player, which instantly moves them to a fixed destination on the other map.
Justification
To let players travel between maps, I created two custom classes. TeleportationCircle is a type of ground tile that stores where it should teleport to. When a player steps on it, the circle creates a TeleportAction that handles the actual teleportation - removing the player from the current map and placing them on the destination map. This design splits up the work: TeleportationCircle manages the ground tile setup, while TeleportAction does the teleporting. This design reduces connascence of execution and timing by keeping teleportation logic contained within the TeleportAction class. The actor doesn't need to know the details of when or how teleportation works - it just gets the option through allowableActions(). The teleport destinations are set up manually in the Application class. This way, only the player can use teleportation, so it won't mess with how NPCs behave. Pros: ● SRP : Each class has one clear job ● OCP : New teleport circles or maps can be added easily without changing existing code. ● The destination information stays contained within each TeleportationCircle object. Cons: ● Each teleport circle only goes to one place. ● All destinations are hardcoded in Application, making it harder to expand the system dynamically.
Alternate Design
Another approach would have been to put the teleportation check and logic directly in the Player class. The player would check each turn if they're on a teleportation tile ('A'), then handle moving between maps. I rejected this because it tightly couples the player's turn logic with map behavior, violating SRP by making the Player class handle movement and map changes. It also breaks OCP since modifying teleportation behavior becomes harder. The current design keeps teleportation logic in dedicated classes (TeleportationCircle and TeleportAction), maintaining loose coupling and organization.

Feature

Creatures like SpiritGoats, OmenSheep, and GoldenBeetles now have different ways of choosing what to do. When we create each creature, we can pick from two options:
Priority-based selection - The creature checks its actions in order of importance, just like the old A1/A2 system.
Random selection - The creature picks a random action each turn. If it can't do that action, it skips its turn entirely(or does nothing).

Justification

I created a SelectBehaviour interface. I created two specific strategy classes: SelectPriorityBehaviour and SelectRandomBehaviour. These get passed into a new base class called SelectableBehaviourCreature, which builds on NonPlayableActor. This way, each creature can choose how it behaves when it's first created.
Once a strategy is set, it stays the same throughout the creature's life, keeping behavior predictable. This means a SpiritGoat using random behavior will act completely different from one using the standard priority system, even though both are the same creature class.

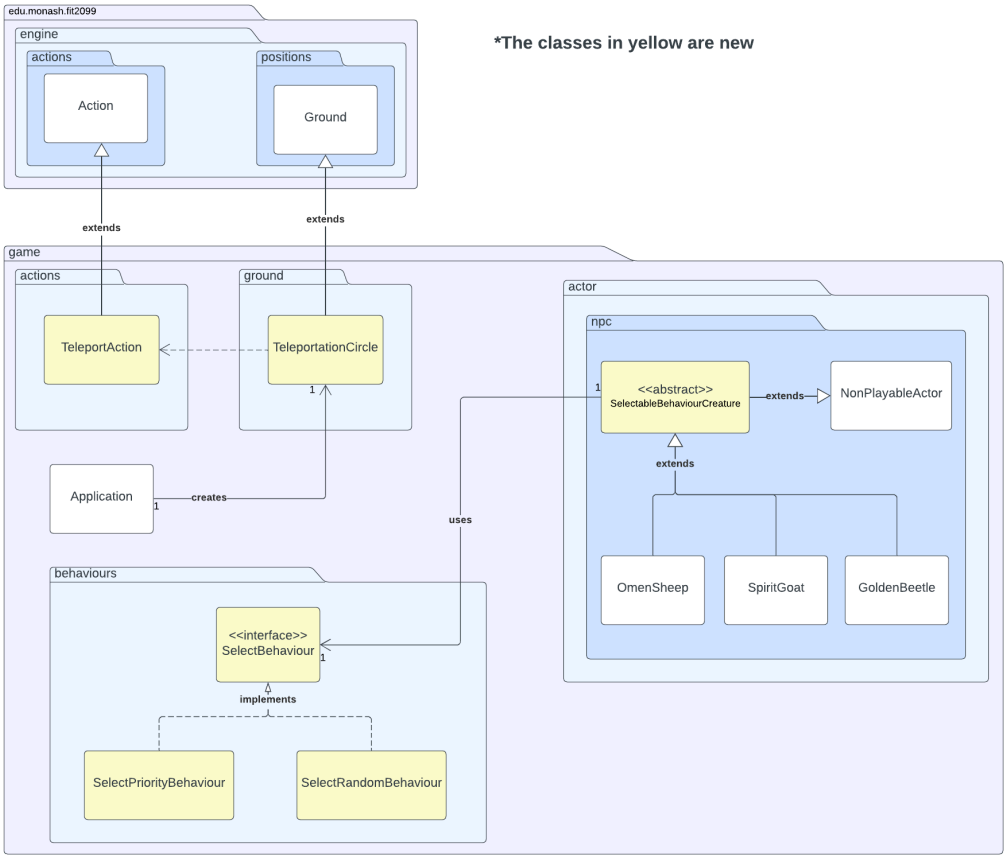
Pros:
- SRP : Behavior selection is kept separate from the main creature code.
- OCP : new strategies can be added without changing existing code.
- LSP : All SelectableBehaviourCreature subclasses can use any SelectBehaviour implementation.
- DRY : Keeps strategy logic in one place instead of duplicating it in different creature classes.

Cons:
- Once set, the strategy(behaviour selection) can't change during the game.
- Uses slightly more memory (each creature stores a behavior object).

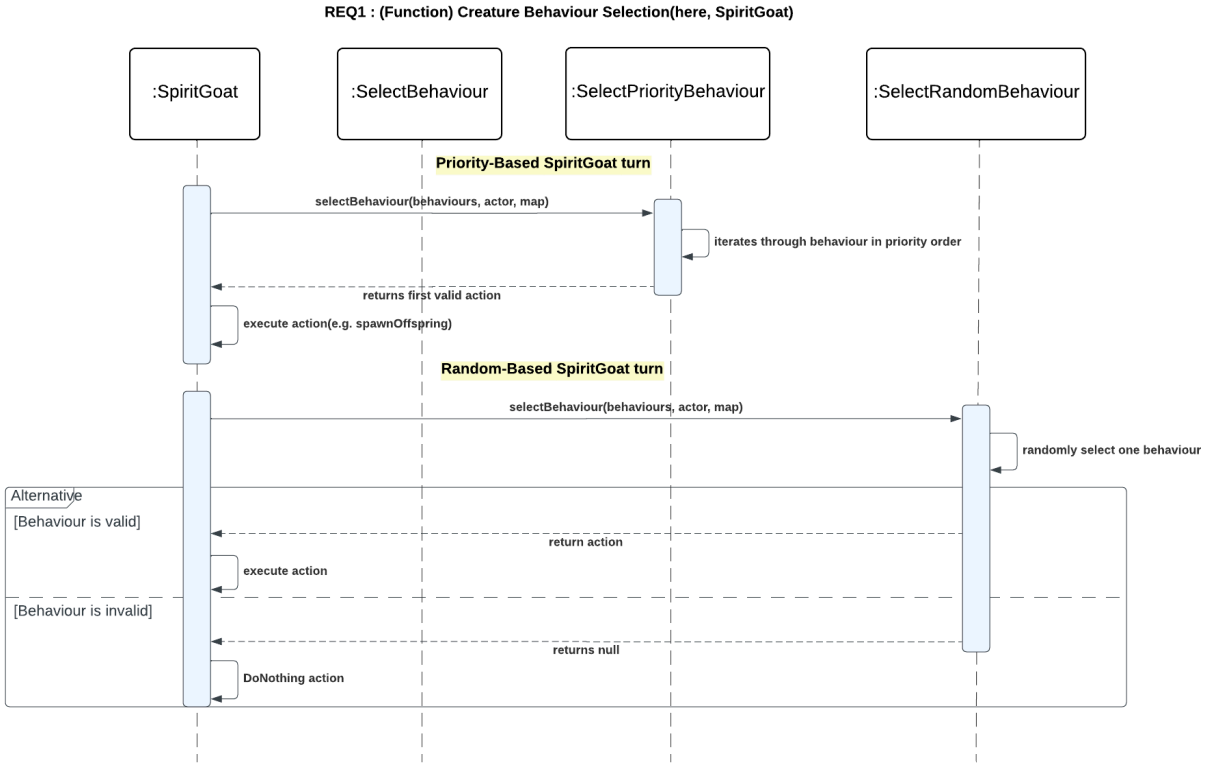
Alternate Design

An alternate approach was to make all creatures use the SelectBehaviour interface directly. However, this would cause problems because some creatures don't need behavior strategies at all. They would still be forced to implement code they don't use, which breaks good design principles. So I chose this approach, which fixes this issue by only using the SelectableBehaviourCreature class for creatures that actually need it.

UML Diagram



Sequence Diagram



REQ 2

Design rationale

Feature

The game introduces a stationary boss called BedOfChaos that either attacks nearby actors or grows additional parts (branches or leaves) when idle. Each turn, if no actor is adjacent, the boss has a 50% chance of growing a Branch or a Leaf. Leaves passively heal the boss each tick, while branches recursively accumulate attack power. This behavior is implemented through the new classes: BedOfChaos, Branch, Leaf, and enhancements to BareFist..

Justification

To satisfy the requirement of a non-wandering boss that grows and attacks based on its parts, a dedicated class BedOfChaos was implemented, extending NonPlayableActor. It overrides playTurn method to either attack adjacent actors or invoke grow() when idle. The grow method appends a new TreePart (either Branch or Leaf) to a maintained list.

The recursive structure of Branch and Leaf allows flexible accumulation of attack power without modifying the core logic. Each Branch may grow its own children (Leaf or Branch), while Leaf overrides tick() to heal the boss each turn.

To ensure the boss's attack reflects its growth, calculateTotalAttack() adds the base damage with the bonuses provided by each TreePart. The BareFist class was modified to support dynamic damage and accuracy through constructor parameters, enabling weapon behavior to adjust based on growth.

Pros:

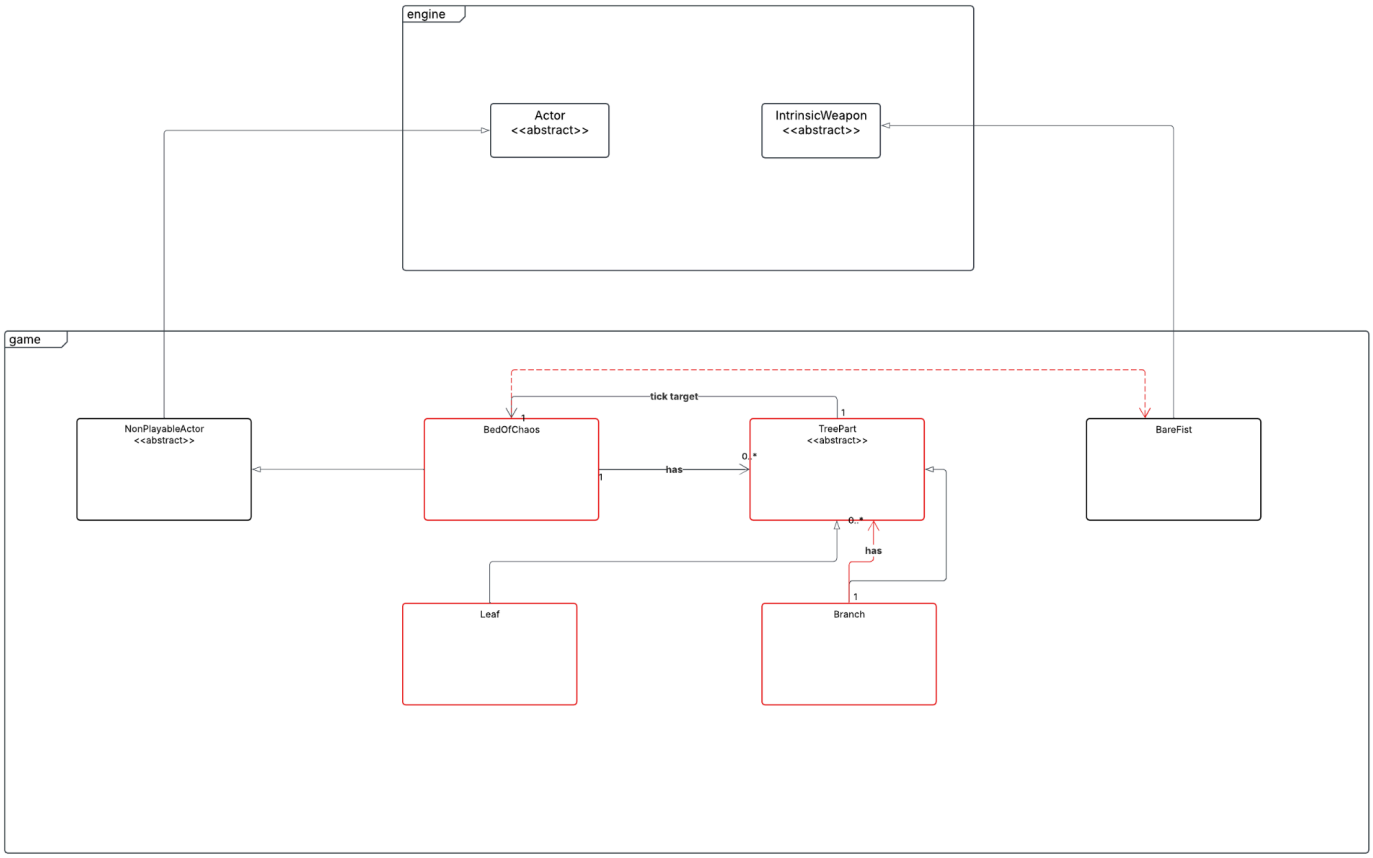
- SRP:
    - BedOfChaos handles coordination and combat logic.
    - Branch and Leaf encapsulate specific behavior such as growth and healing.
- OCP:
    - New parts (Thorn, Fruit) can be added by subclassing TreePart without modifying existing classes.
- LSP:
    - TreePart, Branch, and Leaf are interchangeable from the perspective of the boss.
- Encapsulation:
    - Growth and healing logic are localized within respective classes.

Cons:

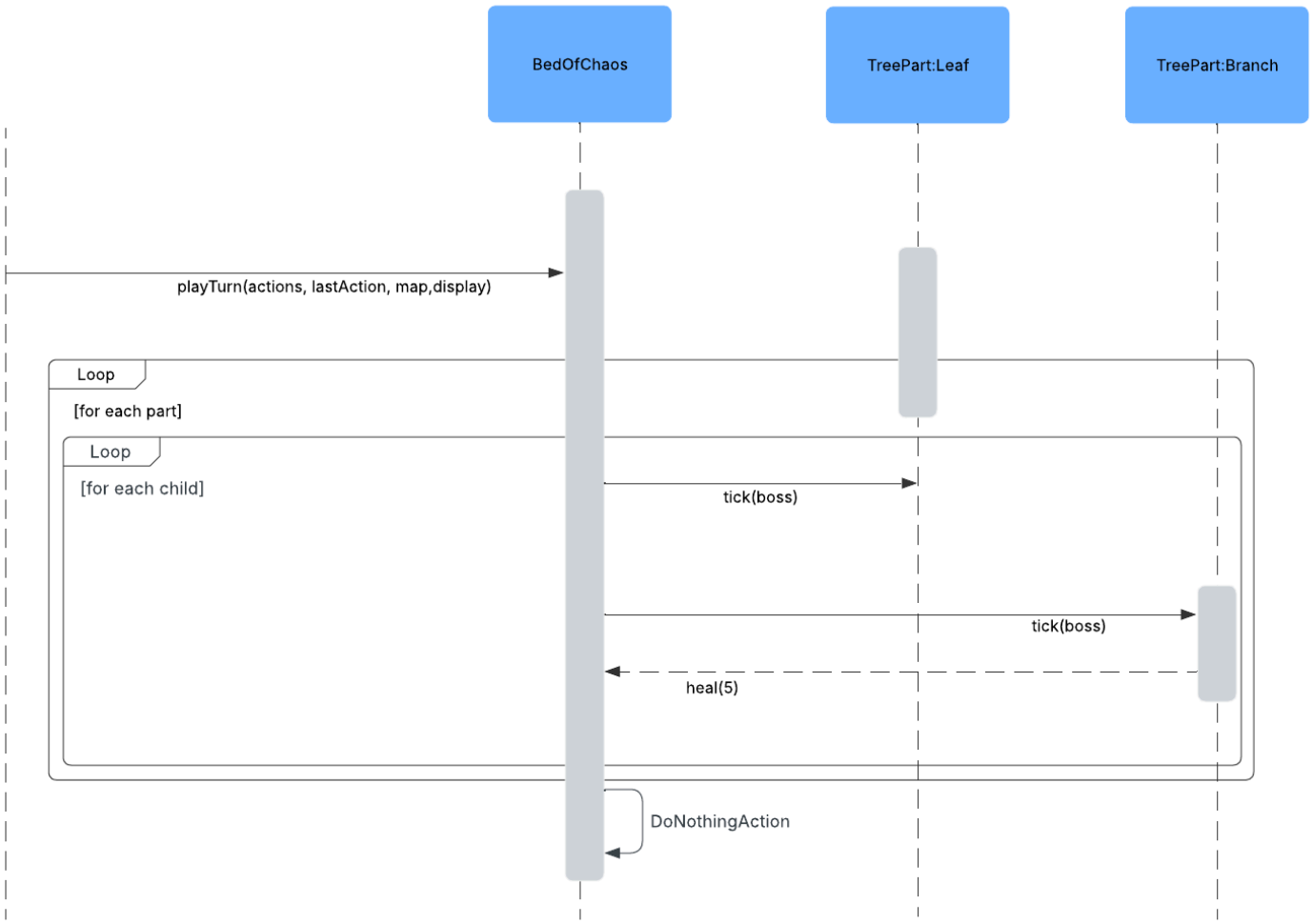
- BedOfChaos updates its intrinsic weapon damage every time it attacks, which may affect readability or predictability.
- Recursive attack computation in branches may introduce complexity if growth becomes deep.
- Healing is not printed explicitly, which may reduce visibility for players during gameplay.

Alternative Design
An approach where BedOfChaos directly tracks integer counts for each part type was considered but rejected due to violation of SRP and OCP. Another alternative using a centralized TreeGrowthManager to manage parts was also avoided due to increased coupling and loss of part-level encapsulation. The chosen implementation uses polymorphism, encapsulation, and composition to maintain clarity, extensibility, and alignment with the SOLID principles.

UML Diagram



Sequence Diagram



REQ 3 - Serena Zhou

Design rationale

Design Choice 1:

Part of my requirement was to include a more realistic farming system. I need to allow the player to first dig a hole, then they will be allowed to plant plants.

In order to implement this, one choice would be to make it so that the ShovelAction be added when the ground detects a shovel in the player's inventory. To do that I could use instance of.

Pros	Cons
Allows us to add the ShovelAction to the player's list of actions	Violates open and closed principle, using instance of every time I would need to add a new class that would allow the player to dig a hole makes this not extendible. You shouldn't need to modify existing code, but rather extend it.
Allows us to check that the Player actually has the shovel in their inventory	Need to add "instance of… OR instance of… OR instance of…" would violate DRY, don't repeat yourself.
	Breaks encapsulation by checking another objects internal type

The second is to add a Capability enum to the shovel, so that whoever possesses the shovel will have that capability. This way the ground can also check if the player possesses the capability to dig a hole.

Pros	Cons
Allows us to not violate SOLID, the open and closed principle. Since the next time we have an item like a shovel to be able to dig holes we can just add the same capability SHOVELABLE. Extending but not modifying	May not be used in the future, redundant
If we needed to modify or change how Shovel is used, it is easy for us to change its implementations without needing to refactor our entire codebase	

Design Choice 2:

For the next part I needed to implement WaterAction, which allows the player to water plants to keep them from dying. Now from the previous design choice I already knew that in order to add a way for us to check if the player has a WateringCan we can use capabilities. However we need to consider the fact that some plants will need to be watered, some don't.

The first choice in order to implement this is you take in a plant in order to water. We check its name in order to water it, adding a new method to the plant interface to reset the water level.

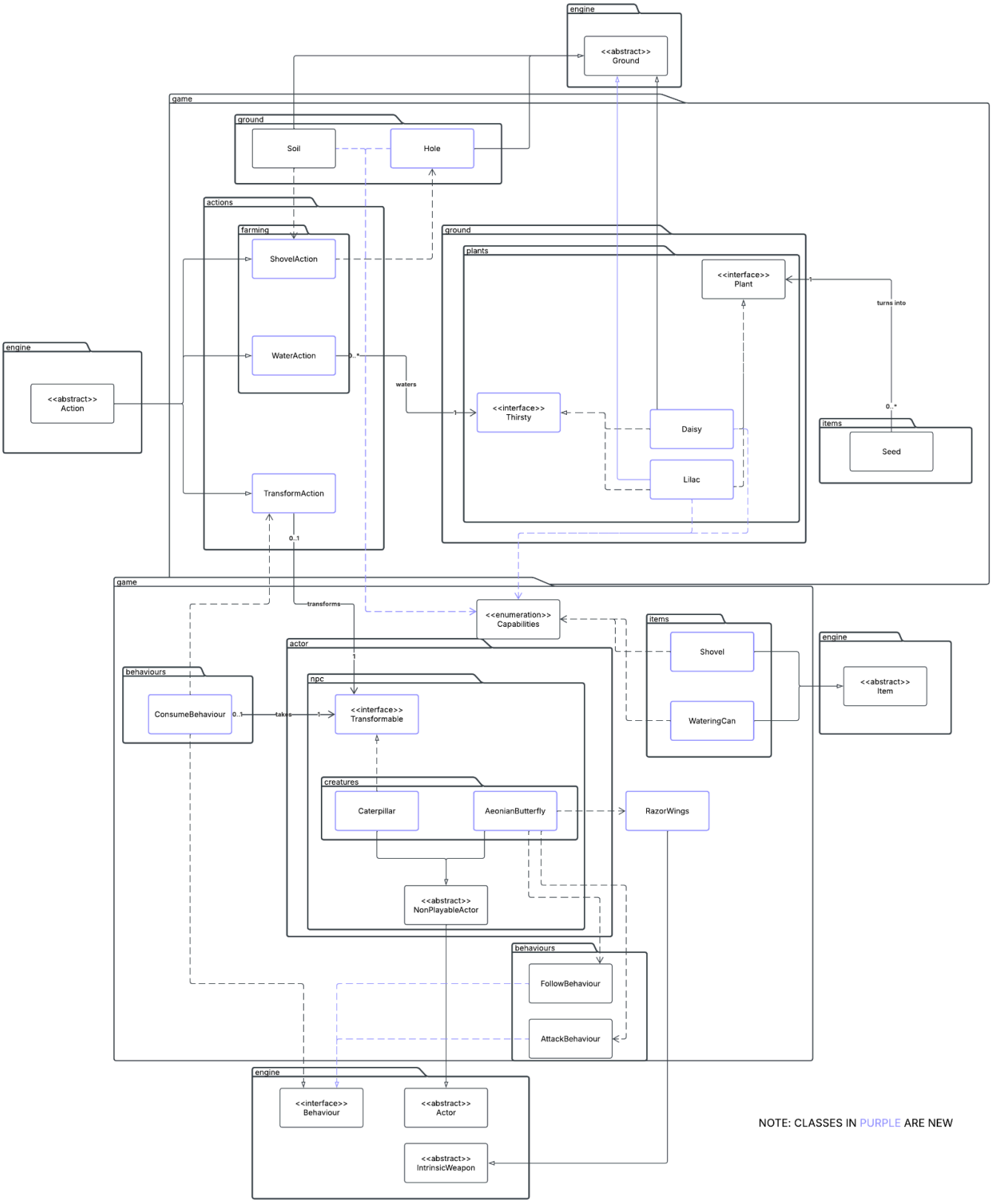
Pros	Cons
Allows us to make sure that the plant we water is the correct plant (Daisy/Lilac)	Violating the open-closed principle, we would be hard-coding and checking strings. This makes it so that our code is not extendible at all, in addition to modifying the code.
	Violates the single-responsibility principle, the plant interface class should be handling making sure that plants are able to be planted. However now you would also be adding the extra responsibility of watering plants, even though certain plants can't even be watered
	Interface segregation principle is also violated, the interface plant is too big, each plant can do certain things (like being watered) but not all of them. Lumping them into the same categories making them implement a method that they don't need is redundant
	Violates the DRY rule, makes it so that if we add a new plant that needs to be watered we would need to add another condition to check its string

The second choice would be to create a new interface called Thirsty and all plants that need to be watered every few turns (are thirsty) would implement this interface. The WaterAction will add a Thirsty plant as its attribute and watering it will give the farmer 100 runes as well as resetting the timer of the plant.

Pros	Cons
Ensures that whatever you are watering, they have the capability of being watered	May be over-engineering if in the future only one or two classes need to be watered.
Prevents violating the open and closed principle by making our code open for extension but closed for modification. Eg if we needed to change the logic for watering the plants, all we need to do is check the 'water' method	
Reduces repeated code, WaterAction knows to only	

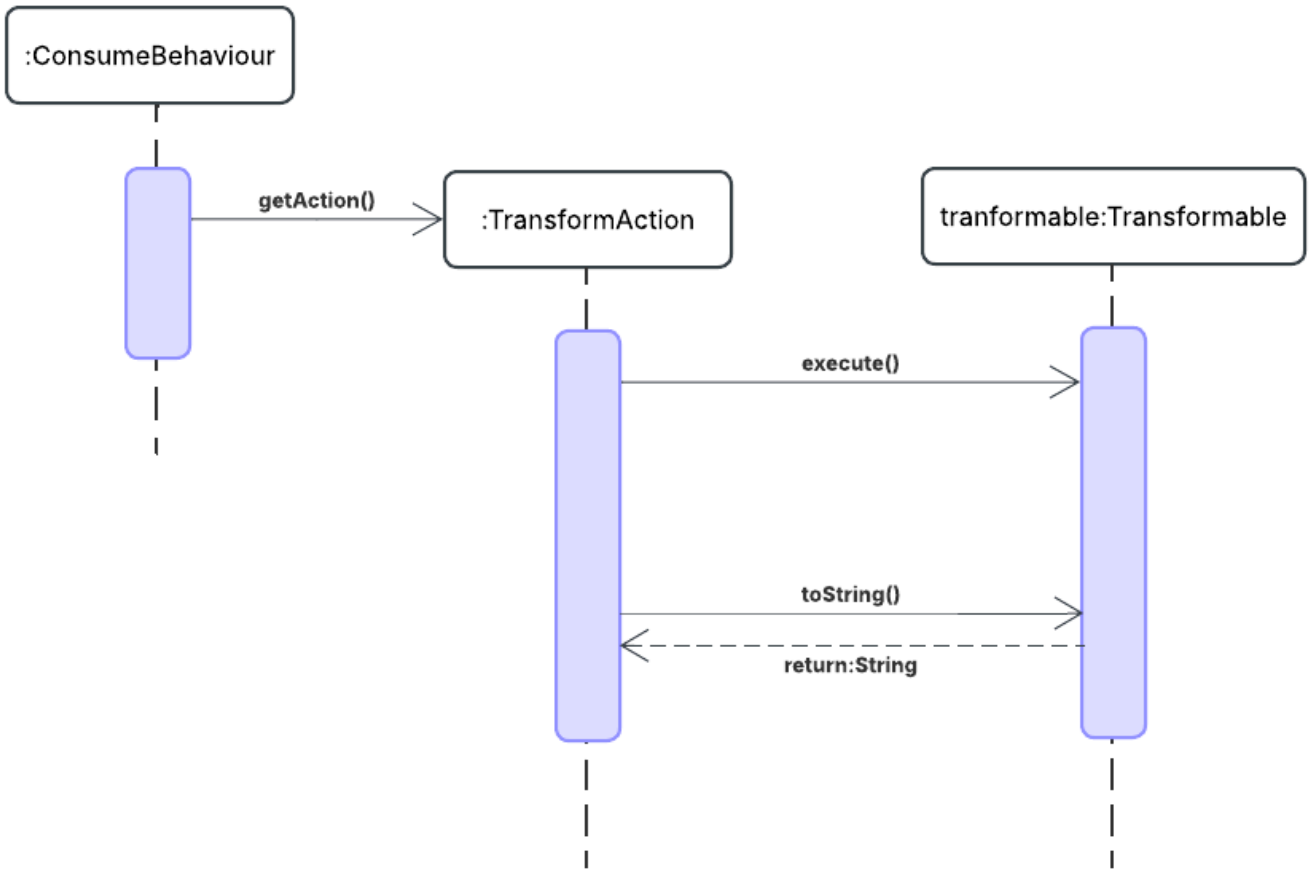
call the water method, does not need condition checks in order to make the code work	
Easy to add new classes that are Thirsty in the future, you just need to let those classes implement Thirsty interface	

UML Diagram - NEW CLASSES ARE IN PURPLE



NOTE: CLASSES IN PURPLE ARE NEW

Sequence Diagram



REQ 4 - Aryan Mobaraki/Sandeesa Rajapaksa

Design rationale

Feature
There will need to be a Faction class (higher level class) - Merchant faction - Hostile faction - Creature faction Your standing with a faction can affect prices or behaviour. - for example if your standing with the merchant faction is above 5, you get 50% discount for items - if your standing with friendly animals is less than 5, they will avoid you (move away when you move close) - conversely if your standing with the Hostile faction is high enough, they will be more likely to move away after attacking you
Justification
For factions, an abstract class was chosen. The reason for this choice is because there may be many factions, but ultimately, they will share the same basic methods and attributes. Having an abstract class boosts extensibility and OCP; any new factions can simply extend the existing faction abstract class. Each faction represents all members. Members of a faction are identified if they have Capabilities.[Faction Specific Capability]. This allows constant time access to a faction through ANY member. This helps maintain adherence to DIP; the high level faction offers a level of abstraction. Any class that interacts with a faction can do so with any faction. For example in the GiveAction class there is an affectFaction(Actor actor) from the Affectionable interface. This supports DIP as this class will be able to execute any effect from any faction (as well as effect the faction itself. The negative side to this approach is that the effect of low/high standing with a faction may be limited (for example in GiveAction you may need to change execute() it slightly if the effect were to be altered). This aspect does not necessarily adhere to OCP, however given the scope of this task it was chosen due to its readability and simplicity.

Alternative Design
An abstract Faction class is probably the best solution here. However adding the effects of doing an action to a member of a faction could be done in an alternative way: Instead of affectFaction(Actor actor) (from Affectionable interface) relying on conditionals one could instead pass Faction IDs (Capabilities.HOSTILE, Capabilities.MERCHANT etc) into a ‘subjects’ list. One could then iterate through the list, and if the actor being acted on is of group Capabilities.[Insert Faction] then enact the effect for THAT PARTICULAR faction. This would boost extensibility (as you can simply append any NEW factions to this list). However since in my requirement I specifically determined that the will only be ‘a total of 3 Factions’, this was not employed. If later new factions were to be added this may be a suitable approach.

Feature
Affection level (labelled as ‘Standing’ in my code).
Justification

For this feature a class called FactionStandingManager was made. This concrete class was made to improve SRP; instead of having each Faction manage thing; 1 class was made for this one job: Keep track of factions and allow other classes to access them.

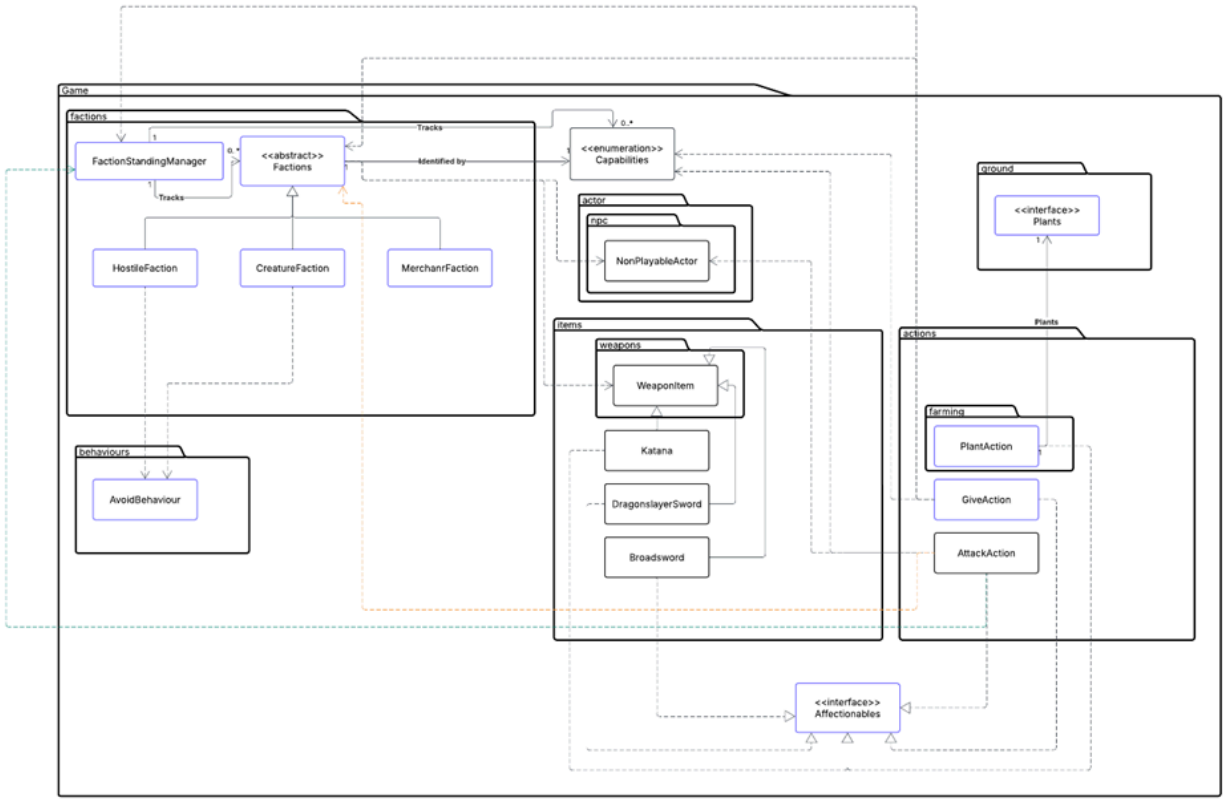
Since the attribute allFactions has the STATIC keyword, it allows it to be accessed easily.

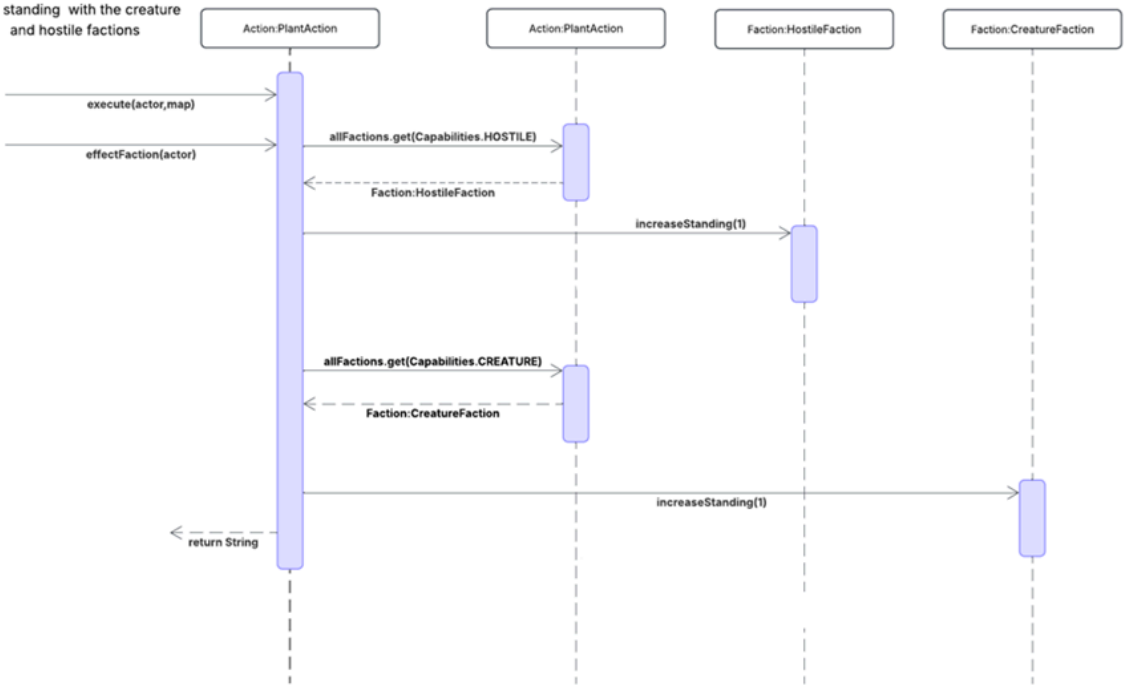
Factions and effects can be modified by updating the allFactions hashmap. This is excellent for enforcing OCP; if a new faction is needed with new effects (or if the MERCHANT faction needs to be replaced) allFactions can simply be updated.

Furthermore, any class which triggers an effect (for example GiveAction may trigger Merchant faction prices to go down) can simply extend the Affectionable interface.

This interface was chosen to prevent ISP, no function is forced to implement effectFaction unless it is required.

UML Diagram
Note: New classes outlines in Blue



Sequence Diagram

This sequence diagram is specific to a senario where the player plants something using plant action, hence increasing their standing with the creature and hostile factions